MarkDownAble

Cet outil transforme un ou plusieurs documents Markdown (`.md`) en :

- **une page web** (HTML) avec sommaire latéral redimensionnable, thème clair/sombre, formules LaTeX, diagrammes Mermaid, code coloré (avec boutons copier / agrandir / télécharger) ;
- **un PDF** avec signets (navigation par titres), en-têtes / pieds de page, sauts de page, thème clair ou sombre.

Il fournit aussi des commandes pour **numéroter les titres** et **générer la table des matières** directement dans le Markdown.

Vous pouvez consulter le dépôt GitHub mais ce n'est pas nécessaire pour utiliser l'outil : https://github.com/benoitdekeyn/MarkDownAble

Par où commencer ?

Votre situation	Où aller
Je découvre l'outil, rien n'est encore installé	§1 Démarrage rapide
C'est installé : je veux écrire et générer mon document	§2 Utilisation au quotidien
Je veux créer un nouveau document	§3 Créer un nouveau document
Je veux savoir ce que je peux écrire (formules, code, Mermaid, sauts de page…)	§4 Écrire dans le Markdown
Je veux changer l'apparence, le PDF, la numérotation ou la table des matières	§5 Personnaliser
Je cherche où sont (ou vont) mes fichiers	§6 Organisation des dossiers
Un message d'erreur ou un avertissement s'affiche	§7 Dépannage

1. Démarrage rapide

De zéro à un aperçu dans le navigateur en quelques minutes. Il faut **Node.js** (obligatoire), **Git** (facultatif) et **Chrome ou Edge** (uniquement pour le PDF).

```powershell
git clone https://github.com/benoitdekeyn/MarkDownAble MonProjet     # remplacez MonProjet
cd MonProjet
npm install
npm run dev                                  # aperçu en direct de doc/exemple.md
```

> **Erreur `npm : Le terme « npm » n'est pas reconnu…` (ou `'npm' is not recognized…`, `npm: command not found`) ?** Node.js (qui fournit `npm`) n'est pas installé, ou le terminal a été ouvert avant l'installation : suivez le §1.1 puis fermez et rouvrez le terminal. De même, `git : Le terme « git » n'est pas reconnu` signifie que Git n'est pas installé : voir le §1.2 (ou téléchargez le dépôt en ZIP).

Le détail, étape par étape :

1.1 Installer Node.js

Node.js est le programme qui exécute l'outil (à faire une seule fois par machine).

1. Allez sur https://nodejs.org et téléchargez la version **LTS** (recommandée).
2. Lancez l'installateur et acceptez les options par défaut.
3. **Fermez puis rouvrez** votre terminal (ou VS Code), puis vérifiez :

   ```powershell
   node -v
   npm -v
   ```

   Chaque commande doit afficher un numéro de version (Node 20 ou plus récent conseillé). `npm` est installé avec Node.js.

1.2 Récupérer le projet

Avec **Git** : `git clone https://github.com/benoitdekeyn/MarkDownAble MonProjet` puis `cd MonProjet`. Git s'installe depuis https://git-scm.com/downloads (options par défaut) ; rouvrez ensuite le terminal et vérifiez avec `git --version`.

Sans Git : sur la page GitHub du dépôt, bouton **Code** → **Download ZIP**, dézippez, puis ouvrez un terminal dans le dossier obtenu.

> Astuce VS Code : clic droit sur le dossier → « Ouvrir dans le terminal intégré ».

1.3 Installer les dépendances de l'outil

Dans un terminal **ouvert dans le dossier du projet** (celui qui contient ce `README.md`) :

```powershell
npm install
```

Cela crée le dossier `node_modules/` (ne le modifiez pas, il n'est pas versionné). À refaire seulement si ce dossier disparaît ou si `package.json` change.

1.4 Voir l'exemple

```powershell
npm run dev
```

Le navigateur s'ouvre sur l'aperçu de `doc/exemple.md` (formules, code, Mermaid, images…). Chaque sauvegarde recharge la page. `Ctrl+C` arrête le serveur.

1.5 Démarrer votre propre document

La méthode la plus simple : une commande crée un `.md` et un `.jsonc` vierges, que vous pouvez écrire aussitôt.

```powershell
npm run init -- MonDoc
npm run dev -- MonDoc
```

Vous pouvez ensuite supprimer l'exemple (`doc/exemple.md` , `doc/exemple.jsonc` et les images `doc/media/exemple_*`). Autre possibilité : dans `doc/` , **renommer** `exemple.md` **et** `exemple.jsonc` **avec le même nouveau nom** (le lien entre les deux se fait uniquement par ce nom), puis remplacer le texte et les images.

Pour les livrables : `npm run edit:outline` (numérotation + table des matières, facultatif), puis `npm run build:html` et/ou `npm run build:pdf` . Les fichiers sont écrits dans `output_html/` et `output_pdf/` . Le détail des commandes est au §2.

1.6 Pour le PDF : Chrome ou Edge

L'export PDF utilise le navigateur déjà installé sur la machine (Google Chrome ou Microsoft Edge, détecté automatiquement). Rien à télécharger de plus ; si la détection échoue, indiquez son chemin avec `pdf.browser` (§5.6).

1.7 Un projet à vous, sans lien avec ce dépôt (facultatif)

Après le clone, supprimez le dossier `.git` (`Remove-Item -Recurse -Force .git` sous PowerShell) puis faites `git init` . Pour recevoir plus tard les améliorations de l'outil, gardez plutôt le dépôt d'origine comme remote (`git remote add outil https://github.com/benoitdekeyn/MarkDownAble` puis `git pull outil main`) et ne modifiez pas `scripts/` .

---

2. Utilisation au quotidien

Toutes les commandes se lancent depuis le terminal, dans ce dossier.

Commande	Effet
`npm run init -- MonDoc`	Crée `doc/MonDoc.md` et `doc/MonDoc.jsonc` (config commentée). Voir §3.
`npm run dev`	Serveur local avec **rechargement automatique** à chaque sauvegarde (ouvre le navigateur sur http://localhost:4173). Ctrl+C pour arrêter.
`npm run build:html`	Écrit `output_html/<nom>/<htmlName>.html` + médias utilisés, et l'ouvre dans l'Explorateur avec `--open`.
`npm run build:pdf`	Écrit `output_pdf/<nom>.pdf` (l'ouvre avec `--open`).
`npm run edit:numbering`	Numérote les titres du `.md` selon `numbering`.
`npm run edit:numbering:guess`	Détecte la numérotation **déjà présente** et réécrit `numbering.levels` dans la config.
`npm run edit:numbering:reset`	Retire toute la numérotation des titres.
`npm run edit:toc`	Génère / met à jour la table des matières dans le `.md`.
`npm run edit:outline`	`edit:numbering` puis `edit:toc`.

Options (à ajouter après `--`) :

```powershell
npm run build:pdf -- exemple          # ne traiter que le document « exemple »
npm run edit:toc -- exemple --dry     # --dry : affiche ce qui serait fait sans rien écrire
npm run build:html -- --open          # ouvre le résultat à la fin du build (rien ne s'ouvre par défaut)
```

- Avec **un seul** document dans `doc/`, le nom est facultatif. Avec **plusieurs**, les commandes `edit:*` exigent le nom ; `build:*` traitent tout si aucun nom n'est donné.
- Variables d'environnement : `MARKDOWN_OPEN=1` (équivaut à `--open`), `MARKDOWN_NO_OPEN=1` (équivaut à `--no-open` : `dev` n'ouvre pas le navigateur), `PORT=5000` (port du serveur `dev`), `MARKDOWN_KEEP_TMP=1` (conserve le HTML temporaire du PDF, utile pour déboguer).
- En mode `dev` avec plusieurs documents, `/` affiche la liste et chaque document est sur `/<nom>/`. Rien n'est écrit dans `output_html/` en mode `dev`.

Lire les messages du terminal

Chaque commande se termine par un **bilan sur les dernières lignes** : ✓ succès, ⚠ avertissements, ✕ erreurs (tous listés avec leur cause). Un indicateur animé s'affiche pendant les générations longues (HTML, PDF). Les couleurs sont automatiques dans un terminal ; `NO_COLOR=1` les désactive, `FORCE_COLOR=1` les force. Un avertissement n'empêche pas la génération ; une erreur indique ce qui n'a pas pu être produit (voir le §7).

Flux de travail conseillé

1. `npm run dev` et rédigez dans votre `.md` de `doc/`.
2. Quand les titres sont stables : `npm run edit:outline` (numérotation + table des matières).
3. `npm run build:html` et `npm run build:pdf` pour les livrables.

---

3. Créer un nouveau document

Une commande crée le Markdown et sa configuration commentée :

```powershell
npm run init -- MonDoc
```

Elle écrit `doc/MonDoc.md` (un squelette, jamais écrasé s'il existe) et `doc/MonDoc.jsonc` (tous les réglages avec leurs explications). Des options préremplissent la config pour aller plus vite :

```powershell
npm run init -- MonDoc --title "Mon document" --icon media/logo.png --theme
dark --pdf-theme light --format A4 --margin 15mm --page-breaks 1,2 --numbering
decimal
npm run init -- --help        # liste complète des options
```

Option	Effet
`--title "…"`, `--icon chemin`, `--language fr`	Titre, icône / logo (image de `doc/`), langue.
`--theme auto|light|dark`, `--pdf-theme light|dark`	Thème de la page HTML et du PDF.
`--html-name`, `--pdf-name`	Noms des fichiers générés (sans extension).
`--embed-images`	Images incorporées au HTML.
`--format A4`, `--landscape`, `--zoom 100`, `--margin 20mm` ou `--margins "20mm 18mm"`	Mise en page du PDF.
`--page-breaks 1,2` (ou `none`), `--hide-toc`, `--no-header`, `--no-footer`	Sauts de page, TOC, en-tête / pied du PDF.
`--numbering off|decimal`, `--toc-range 2,3`	Numérotation des titres, niveaux de la table des matières.
`--no-md`, `--force`	Ne pas créer le `.md` ; écraser un `.jsonc` existant.

Vous pouvez aussi le faire à la main : créez `doc/MonDoc.md` et `doc/MonDoc.jsonc` (même `{}` suffit, tout a une valeur par défaut).

Chaque document a son propre dossier `output_html/<nom>/`, donc plusieurs documents peuvent garder `htmlName: "index"` sans conflit. Seuls deux **PDF** de même nom entrent en conflit (l'outil s'arrête alors en le signalant : changez `pdf.name`).

---

4. Écrire dans le Markdown

- **Titres, tableaux, listes de tâches, liens internes** `[voir](#titre-de-la-section)` comme sur GitHub (ancres générées comme GitHub).
- **Liens vers un autre fichier** : les chemins sont relatifs au `.md` (donc `media/…`, pas `doc/media/…`).
- **Formules LaTeX** : `$x^2$` en ligne, `$$ \int_0^1 f(x)\,dx $$` en bloc.
- **Code coloré** : `c</code>, <code>python`, etc. Chaque bloc a une barre avec le nom du langage ; dans la page HTML elle propose aussi **copier**, **agrandir** (quasi plein écran, `Échap` pour fermer) et **télécharger** (extension déduite du langage, `.txt` sinon). Dans le PDF, seule l'étiquette du langage apparaît.
- **Mermaid** : bloc ```` ```mermaid ````. Options par diagramme sur la même ligne :

  markdown

  ````
  ```mermaid layout=elk direction=LR
  graph TD
    A ⟶ B
  ```
  ````

  `layout` : `dagre` (hiérarchique) ou `elk` (adaptatif). `direction` : `TB`, `BT`, `LR`, `RL` (flowchart / graph / stateDiagram).
- **Saut de page (PDF uniquement)** : une ligne `\newpage` ou `<!— newpage —>` (ignoré à l'écran). Il fonctionne aussi dans une liste, au milieu d'un paragraphe, dans la TOC, et **dans un tableau Markdown** : une ligne `\newpage` entre deux lignes du tableau n'est pas affichée et la ligne suivante commence une nouvelle page.
- **Annuler un saut automatique** : `\samepage` (ou `<!— samepage —>`) **juste avant un titre** dont le niveau figure dans `pdf.pageBreakBefore` supprime le saut de page imposé pour ce titre seul. Il est ignoré à l'écran ; placé ailleurs que devant un titre, il est signalé comme erreur.
- **Page sans en-tête ni pied de page** (page de garde…) : `<!— nohf —>` (ou `\nohf`) en **toute première ligne** du document, ou **juste après** un saut de page.

- **Sauts de page automatiques** : `pdf.pageBreakBefore: [1, 2]` (niveaux de titres) ; `keepSectionsTogether` / `keepBlocksTogether` évitent de couper sections et blocs quand ils tiennent sur une page.
- **En-têtes / pieds de page PDF** : variables `{page}`, `{pages}`, `{title}`, `{date}`, `{h1}` … `{h6}`, HTML autorisé (logo `<img src="media/logo.png">`).

---

5. Personnaliser : le fichier `.jsonc` et le dossier `template/`

5.1 Principes

Chaque document `doc/<nom>.md` est piloté par `doc/<nom>.jsonc`. C'est du **JSON qui accepte les commentaires** (`// …`), donc :

- les clés et les textes sont entre guillemets `"…"`, les valeurs sont séparées par des virgules, **sans virgule après le dernier élément** d'un bloc ;
- les nombres s'écrivent sans guillemets (`100`), les booléens `true` / `false`, « rien » s'écrit `null` ;
- les chemins utilisent `/` (ex. `"media/logo.png"`) et sont **relatifs à `doc/`** ;
- **toute clé omise prend sa valeur par défaut** : un fichier `{}` est valide, et vous ne notez que ce que vous changez ;
- une valeur invalide (ou une clé inconnue dans `numbering`, `toc`, `header` / `footer`) arrête la génération avec un message qui indique la clé fautive ; une clé inconnue ailleurs est simplement ignorée.

Squelette complet (valeurs par défaut de l'outil, sauf indication) :

jsonc

```jsonc
{
  "htmlName": "index",          // → output_html/<nom>/index.html
  "title": null,                // null = nom du .md
  "icon": null,                 // ex. "media/logo.png"
  "language": "fr",
  "theme": "auto",              // auto | light | dark
  "embedImages": false,

  "numbering": { "levels": { /* §5.4 */ } },
  "toc":       { "range": [2, 3], "collapsible": false, "summary": "Table des
matières", "open": false, "levels": { /* §5.5 */ } },
  "pdf":       { /* §5.6 */ },
  "layout":    { "sidebarWidth": 320, "sidebarMinWidth": 200,
"sidebarMaxWidth": 700, "contentWidthPercent": 90 },
  "style":     { /* §5.7 */ }
}
```

> Le fichier doc/exemple.jsonc est un exemple complet et commenté. `npm run init` génère le même type de fichier (§3).

5.2 Général : fichiers de sortie, titre, icône, thème

Clé	Type / valeurs	Défaut	Effet
`htmlName`	texte sans extension	`"index"`	Nom du HTML : `output_html/<nom>/<htmlName>.html`. Ni `/`, `\`, `:`, `*`, `?`, `"`, `<`, `>`, `|`. Un `.html` écrit par erreur est retiré.
`title`	texte ou `null`	`null`	Titre de l'onglet, du sommaire latéral et variable `{title}` des en-têtes PDF. `null` = nom du `.md`.
`icon`	chemin ou `null`	`null`	Image **de** `doc/` utilisée comme favicon et comme logo devant le titre du sommaire (HTML / `dev`, pas le PDF). Copiée dans `output_html/<nom>/` (ou incorporée avec `embedImages`). Fichier introuvable ou hors de `doc/` → avertissement.
`language`	code langue	`"fr"`	Attribut `lang` de la page et format de `{date}` dans le PDF (`"fr"` → `28/10/2026`, `"en"` → `10/28/2026`).
`theme`	`"auto"`, `"light"`, `"dark"`	`"auto"`	Mode initial de la page HTML (clair/sombre), pas le thème de coloration du code. `auto` suit le système. Le bouton de la page change de mode et mémorise le choix dans le navigateur. N'affecte pas le PDF (voir `pdf.theme`).
`embedImages`	`true` / `false`	`false`	`true` : images en base64 dans le HTML, un seul fichier autonome mais lourd, rien à copier. `false` : HTML léger, images copiées dans `output_html/<nom>/`.

jsonc

```jsonc
{
  "htmlName": "document-final",   // output_html/<nom>/document-final.html
  "title": "Projet – Maison intelligente",
  "icon": "media/logo.png",
  "theme": "dark",
  "embedImages": true             // un seul fichier à envoyer par mail
}
```

Les noms de PDF se règlent dans `pdf.name` (§5.6).

5.3 Disposition de la page HTML : `layout`

Clé	Défaut	Effet
`sidebarWidth`	`320`	Largeur initiale du sommaire latéral, en px. L'utilisateur peut la changer en glissant la bordure (double-clic : retour à cette valeur) ; son choix est mémorisé.
`sidebarMinWidth` / `sidebarMaxWidth`	`200` / `700`	Bornes de ce redimensionnement, en px.
`contentWidthPercent`	`90`	Part de la largeur **restante** (à droite du sommaire) occupée par le texte.

jsonc

```jsonc
"layout": { "sidebarWidth": 380, "contentWidthPercent": 80 }
```

5.4 Numérotation des titres : `numbering`

Commandes concernées : `npm run edit:numbering`, `edit:numbering:guess`, `edit:numbering:reset`. **Le `.md` n'est modifié que quand vous lancez ces commandes.**

`numbering.levels` contient une entrée par niveau (`"1"` = `#`, `"2"` = `##`, … `"6"` = `######`). Chaque niveau accepte :

Clé	Valeurs	Effet
`style`	`ignore`, `none`, `decimal`, `alpha`, `ALPHA`, `roman`, `ROMAN`	Type de numérotation (tableau ci-dessous).
`separator`	texte	Texte écrit **après chaque indice** : `"."`, `" "`, `"–"`, `" – "`… Avec `none`, c'est le symbole écrit devant le titre.
`cumulative`	`true` / `false`	`true` : inclut les indices des parents (`2.1.3.`). `false` : indice du niveau seul (`3.`).

`style`	Exemple de rendu (séparateur `"."`)
`ignore`	Niveau **jamais touché** : l'outil ne l'écrit ni ne l'efface. Idéal pour les niveaux que vous écrivez à la main.
`none`	Aucun numéro. Sans `separator`, l'éventuelle ancienne numérotation / ancien symbole est **retiré**. Avec `separator: "▪"`, chaque titre commence par ce symbole (`## ▪ Titre`).
`decimal`	`1.` `2.` `3.`
`alpha` / `ALPHA`	`a.` `b.` … / `A.` `B.` …
`roman` / `ROMAN`	`i.` `ii.` `iii.` … / `I.` `II.` `III.` …

Exemple : parties en chiffres romains, sections en décimal cumulatif, sous-sections avec un symbole, le reste ignoré.

jsonc

```jsonc
"numbering": {
  "levels": {
    "1": { "style": "ignore" },
    "2": { "style": "ROMAN",   "separator": "." },                          //
## I. Introduction
    "3": { "style": "decimal", "separator": ".", "cumulative": true },     //
### 1.1. Contexte  (sous « I. »)
    "4": { "style": "none",    "separator": "▪" },                          //
#### ▪ Détail
    "5": { "style": "ignore" },
    "6": { "style": "ignore" }
  }
}
```

Résultat dans le `.md` après `npm run edit:numbering` :

markdown

```markdown
## I. Introduction
### 1.1. Contexte
#### ▪ Détail
```

Comment ça se passe :

- **Changer de schéma** : modifiez `levels`, relancez `npm run edit:numbering`. L'ancienne numérotation est retirée automatiquement, grâce à la mémoire `scripts/state/<nom>.numbering.json` (dernier schéma appliqué).
- **L'éditeur a perdu le fil** (mémoire absente, titres modifiés à la main) : lancez `npm run edit:numbering:guess`. L'outil détecte la numérotation **réellement présente** dans le `.md` (niveau par niveau, majorité ≥ 50 %), **réécrit le bloc** `numbering.levels` du `.jsonc` pour la refléter (les commentaires placés dans ce bloc sont perdus) et la mémorise. Adaptez ensuite `levels` et lancez `edit:numbering`.
- **Tout enlever** : `npm run edit:numbering:reset` (après un `guess` si besoin). Le `.jsonc` n'est pas modifié.
- Les liens internes `[texte](#ancre)` sont **mis à jour** quand les titres changent.
- Sans bloc `numbering` dans le `.jsonc`, les niveaux 2 et 3 sont numérotés en `decimal` cumulatif et les autres ignorés. Les fichiers créés par `npm run init` écrivent explicitement `ignore` partout (rien n'est numéroté tant que vous ne le décidez pas).

- Limite : un titre numéroté dont le texte commence lui-même par un nombre (`2024 bilan`) peut perdre ce nombre. Utilisez `--dry` pour prévisualiser : `npm run edit:numbering -- --dry`.

5.5 Table des matières Markdown : `toc`

Commande : `npm run edit:toc` (ou `edit:outline` = numérotation puis TOC). La table est écrite dans le `.md` **entre les repères** `<!-- TOC -->` et `<!-- /TOC -->` :

- première fois (pas de repères) : insérée **tout en haut** du document (après d'éventuels commentaires de tête comme `<!-- nohf -->`) ;
- ensuite : mise à jour **à l'endroit où se trouvent les repères**, même si vous les avez déplacés ;
- elle n'est jamais régénérée automatiquement à l'enregistrement.

> À ne pas confondre avec `style.toc` (§5.7), qui règle l'apparence du **sommaire latéral** de la page HTML.

Clé	Défaut	Effet
`range`	`[2, 3]`	`[niveau min, niveau max]` des titres listés. Un titre d'un niveau **exclu au-dessus** du min disparaît, et ses enfants sont listés à la suite, au même rang. Ex. `[2, 3]` avec plusieurs `#` : toutes les sections `##` de tous les `#` sont listées d'une traite.
`collapsible`	`false`	`true` : la table entière est repliable (`<details>`) sous le libellé `summary`. HTML et Markdown/GitHub seulement, **toujours dépliée dans le PDF**.
`summary`	`"Table des matières"`	Libellé cliquable quand `collapsible` est `true`.
`open`	`false`	`true` : les blocs repliables sont ouverts par défaut.
`collapsibleLevels`	`null`	Tableau des niveaux dont l'entrée se replie sur ses sous-entrées, ex. `[2]` ou `[2, 3]` (`[]` = aucun). Chaque niveau est indépendant. S'il est fourni, il **l'emporte** sur les `collapsible` de `levels`.

Clé	Défaut	Effet
`foldOffset`	`2`	Alignement avec repli : la flèche ▶ pousse le texte des entrées repliables vers la droite ; les autres entrées reçoivent ce retrait supplémentaire (en demi-cadratins) pour rester alignées. À ajuster selon l'afficheur (GitHub, VS Code, page HTML) : augmentez si les entrées repliables sont plus à droite, diminuez dans le cas inverse. `0` = aucune compensation.
`levels`	voir ci-dessous	Mise en forme par niveau (`"1"` … `"6"`).

> **Quand le changement s'applique-t-il ?** Uniquement quand vous relancez `npm run edit:toc` : la table est réécrite sur place entre les repères (repli ajouté **ou retiré**, rien à nettoyer à la main). `npm run dev` et `build:*` ne la régénèrent jamais ; ils affichent ce qui est déjà dans le `.md`.
>
> **Rendu des replis.** Dès qu'un repli est activé, la table est écrite en HTML compact (sans ligne vide, liens `<a>`), ce qui évite les grands espaces entre entrées. Les marges et la flèche ▶ des `<details>` restent à la charge de l'afficheur (GitHub, aperçu VS Code, notre page HTML). Pour la page HTML de ce générateur, `template/style.css` permet de les ajuster ; ailleurs, le style `size` est ignoré par GitHub.

Options de chaque niveau de `levels` :

Clé	Défaut	Effet
`symbol`	`•` `•` `◦` `–` `·` `·` (niv. 1 à 6)	Texte devant l'entrée (`""` = aucun).
`indent`	`0, 0, 4, 8, 12, 16`	Retrait à gauche, en demi-cadratins (entier de 0 à 60) ; **0 = pas de retrait**.
`bold` / `italic`	gras pour les niveaux 1 et 2 / non	Mise en forme du texte.
`size`	`null`	Taille en em (0.3 à 4). `null` = taille normale. **GitHub ignore la taille** ; ce générateur l'applique.

Clé	Défaut	Effet
`collapsible`	`false`	`true` : cette entrée se replie sur ses sous-entrées (blocs `<details>` imbriqués). HTML/Markdown seulement. Raccourci : `toc.collapsibleLevels`.

Exemple : TOC repliable, niveau 2 en gras avec puce, niveau 3 en italique et plus grand retrait, niveau 4 sans symbole.

jsonc

```jsonc
"toc": {
  "range": [2, 4],
  "collapsible": true,
  "summary": "Sommaire",
  "open": false,
  "levels": {
    "2": { "symbol": "▸", "indent": 0, "bold": true,  "italic": false, "size": 1.1,  "collapsible": true },
    "3": { "symbol": "•", "indent": 4, "bold": false, "italic": true,  "size": null, "collapsible": false },
    "4": { "symbol": "",  "indent": 8, "bold": false, "italic": false, "size": 0.9,  "collapsible": false }
  }
}
```

Seuls les niveaux que vous écrivez sont modifiés : les autres gardent leurs valeurs par défaut. Pour la masquer **dans le PDF uniquement** : `pdf.hideToc` (§5.6).

5.6 Export PDF : `pdf`

Commande : `npm run build:pdf` (Chrome ou Edge requis). Le PDF contient les **signets** (volet de navigation du lecteur PDF) correspondant à tous les titres, et reprend les styles de `style` (polices, couleurs, titres, code, Mermaid) dans le thème `pdf.theme`.

Clé	Défaut	Valeurs / effet
`name`	`null`	Nom du fichier dans `output_pdf/`, sans extension. `null` = nom du `.md`. Deux documents ne peuvent pas avoir le même PDF.
`theme`	`"light"`	`"light"` ou `"dark"`. Le fond sombre couvre aussi les marges.
`format`	`"A4"`	`A3`, `A4`, `A5`, `Letter`, `Legal`, `Tabloid` (insensible à la casse).
`landscape`	`false`	`true` : orientation paysage.
`zoom`	`100`	Pourcentage de 10 à 200. `100` = taille normale ; `80` rétrécit tout, `120` agrandit (moins de contenu par page).
`margins`	`20mm` haut/bas, `18mm` gauche/droite	`{ "top", "right", "bottom", "left" }`, chacun avec une unité : `mm`, `cm`, `in`, `pt` ou `px`. L'en-tête vit dans la marge du haut, le pied dans la marge du bas : prévoyez au moins ~15 mm si vous en utilisez.
`pageBreakBefore`	`[]`	Niveaux de titres qui **commencent une nouvelle page** : `[1, 2]` = chaque `#` et chaque `##`. Les niveaux **non listés** (ex. `###`) peuvent partager la même page. `[]` = aucun saut automatique.
`hideToc`	`false`	`true` : retire du PDF la table des matières Markdown (entre `<!-- TOC -->` et `<!-- /TOC -->`) et son titre.
`keepSectionsTogether`	`true`	Une section (d'un titre au suivant, tous niveaux) qui **tient sur une page** n'est jamais coupée : elle passe en entier à la page suivante (au prix de blanc en fin de page). `false` : remplissage maximal. Une section plus longue qu'une page est évidemment coupée.
`keepBlocksTogether`	`true`	Paragraphes, éléments de liste, blocs de code, tableaux… ne sont **jamais coupés entre deux pages** (sauf s'ils dépassent une page).

Clé	Défaut	Valeurs / effet
`browser`	`null`	Chemin d'un `chrome.exe` / `msedge.exe` si la détection automatique échoue. Ex. `"C:/Program Files/Google/Chrome/Application/chrome.exe"`.
`header` / `footer`	en-tête : `null` ; pied : `{ "center": "{page} / {pages}" }`	En-tête / pied de page (ci-dessous). `null` = désactivé.

En-têtes et pieds de page. Chacun est un objet avec trois zones `left`, `center`, `right` (toutes facultatives) et un `style` CSS facultatif. Les zones acceptent du **HTML** et des variables :

Variable	Valeur
`{page}` / `{pages}`	Numéro de la page / nombre total de pages.
`{title}`	Valeur de `title` (ou nom du `.md`).
`{date}`	Date de génération, au format de `language`.
`{h1}` … `{h6}`	Titre de niveau N : celui qui **commence sur la page**, sinon celui **en cours au début de la page**. Vide s'il n'y en a pas encore.

jsonc

```jsonc
"pdf": {
  "name": "Document-final",
  "theme": "light",
  "format": "A4",
  "zoom": 100,
  "margins": { "top": "22mm", "right": "18mm", "bottom": "20mm", "left":
"18mm" },
  "pageBreakBefore": [1, 2],
  "hideToc": false,
  "header": {
    "left":  "<img src=\"media/logo.png\" style=\"height:8mm\">",    // logo
(chemin relatif à doc/)
    "center": "{title}",
    "right": "{h1} – {h2}",                                          // chapitre
et section en cours
    "style": "font-size:9pt; border-bottom:1px solid #999"          // CSS de
la barre
  },
  "footer": { "left": "{date}", "right": "Page {page} / {pages}" }
}
```

Remarques :

- Les images des en-têtes / pieds sont **incorporées** au PDF ; un chemin introuvable donne un avertissement.
- **Pas d'en-tête sur une page** (page de garde, annexe…) : écrivez `<!-- nohf -->` (ou `\nohf`) **en toute première ligne** du `.md` (pour la 1re page) ou **juste après** un saut de page (pour une autre). Ailleurs, il est ignoré avec un avertissement.
- Ces réglages sont **indépendants** du HTML : `theme`, `layout` et `embedImages` ne concernent pas le PDF, et inversement.

5.7 Apparence : `style`

Ce bloc agit sur la page HTML **et** sur le PDF (sauf mention contraire).

Clé	Défaut	Effet
`fontFamily`	police système (Segoe UI, Roboto, Arial…)	Police du texte, syntaxe CSS `font-family`.
`codeFontFamily`	`ui-monospace, 'Cascadia Code', Consolas…`	Police du code.
`fontSize`	`"16px"`	Taille de base du texte.
`codeFontSize`	`"85%"`	Taille du code, relative au texte.
`pageTheme`	`{ "light": "default", "dark": "default" }`	Palette de couleurs de la page choisie séparément pour le mode clair et le mode sombre.
`pagePalettes`	`{}`	Palettes de page personnalisées, définies uniquement avec des couleurs.
`codeTheme.mode`	`"follow"`	Mode de couleur propre au code : `follow`, `auto`, `light` ou `dark`.
`highlight`	`{ "light": "default", "dark": "default" }`	Thème de coloration syntaxique des balises de code, choisi séparément pour les modes clair et sombre du code.
`mermaidTheme`	`"auto"`	`auto` (suit le thème de la page), `default`, `dark`, `forest`, `neutral`.
`mermaid`	`{ "layout": "dagre", "direction": null }`	Réglages **par défaut** des diagrammes Mermaid (surchargeables diagramme par diagramme, §4).
`headings`	voir ci-dessous	Taille / retrait / couleur de chaque niveau de titre du document.
`toc`	voir ci-dessous	Idem pour les liens du **sommaire latéral** (HTML).
`colors`	`{ "light": {}, "dark": {} }`	Palette (clés ci-dessous), par thème.

Clé	Défaut	Effet
`customCss`	`null`	Chemin (relatif à `doc/`) d'un CSS supplémentaire appliqué **en dernier**, propre à ce document.

Distinguer les deux modes et les deux palettes

Il y a **deux choix distincts** :

1. `theme` **à la racine** décide du mode de la page : `auto` (suit le système), `light` (clair) ou `dark` (sombre). Sur la page HTML, le bouton permet aussi de passer entre ces modes.
2. `style.codeTheme.mode` décide du mode des couleurs de code :
   - `follow` : suit le mode actuellement affiché par la page, bouton compris (conseillé) ;
   - `auto` : suit directement le mode clair/sombre du système, indépendamment du choix de la page ;
   - `light` ou `dark` : impose ce mode au code.

Les palettes associées sont elles aussi séparées :

- `style.pageTheme.light` / `dark` sélectionnent les couleurs générales de page pour chaque mode ;
- `style.highlight.light` / `dark` sélectionnent les couleurs de coloration syntaxique du code pour chaque mode de code.

Voici un exemple complet : la page utilise une palette formelle en mode clair et une palette néon en mode sombre ; le code suit le mode de la page et utilise ses deux feuilles CSS personnalisées.

jsonc

```jsonc
"theme": "auto",
"style": {
  "pageTheme": { "light": "formal", "dark": "neon" },
  "pagePalettes": {},
  "codeTheme": { "mode": "follow" },
  "highlight": {
    "light": "highlight-formal.css",
    "dark": "highlight-neon.css"
  }
}
```

Les palettes de page intégrées sont `default` (mode sombre neutre, gris/noir/blanc), `formal` (clair) et `neon` (sombre coloré). Pour créer une palette de page, déclarez-la dans `pagePalettes`, puis sélectionnez son nom dans `pageTheme` :

```jsonc
"pageTheme": { "light": "papier", "dark": "neon" },
"pagePalettes": {
  "papier": {
    "bg": "#fffdf7",
    "fg": "#262626",
    "accent": "#315a78",
    "sidebarBg": "#f2efe7"
  }
}
```

Les couleurs disponibles sont `bg`, `fg`, `muted`, `border`, `sidebarBg`, `accent`, `codeBg` et `quote`. Les couleurs non indiquées gardent leur valeur par défaut. La palette du code peut utiliser `default` (thèmes intégrés GitHub clair/sombre), le nom d'un thème `highlight.js` (`vs`, `vs2015`, `github-dark`, `nord`…) ou un fichier `.css` placé dans `template/` ou `doc/`. Un thème de code ne change que les couleurs de syntaxe, pas la police ni la forme du bloc ; les fichiers personnalisés de l'exemple sont `template/highlight-formal.css` et `template/highlight-neon.css`.

Les palettes `default`, `formal` et `neon` changent **les couleurs uniquement**. Polices, tailles, coins arrondis et autres choix de présentation restent aux valeurs par défaut dans l'exemple. Il n'y a pas encore de sélecteur JSONC de thèmes complets pour ces réglages ; leur personnalisation globale passe par les fichiers CSS de `template/`.

Mermaid : `layout` et `direction`.

`layout`	Effet
`"dagre"`	« Hiérarchique » : le moteur historique de Mermaid.
`"elk"`	« Adaptatif » : plus compact et lisible sur les gros schémas ; ajoute ~2,4 Mo au HTML.

`direction` : `null` (celle écrite dans le diagramme), `"TB"` (haut → bas), `"BT"`, `"LR"` (gauche → droite), `"RL"`. Seuls `flowchart` / `graph` et `stateDiagram` acceptent une direction.

```jsonc
"mermaid": { "layout": "elk", "direction": "TB" }
```

Titres et sommaire : `headings` et `toc`. Pour chaque niveau `"1"` à `"6"` : `size` (taille, en **em**, 0 à 20 ; 1 = taille du texte ; dans `toc` l'em vaut 14 px), `indent` (retrait à gauche en em, 0 à 20), `color` (`null`, une couleur CSS `"#c0392b"`, ou `{ "light": "…", "dark": "…" }` pour une couleur par thème).

Valeurs par défaut :

Niveau	`headings.size`	`toc.size`	`toc.indent`
1	2	1.1	0
2	1.5	1	0
3	1.25	1	1.5
4	1	0.93	2.5
5	0.875	0.93	3.5
6	0.85	0.93	4.5

(`headings.indent` vaut 0 partout, toutes les couleurs `null`.) Vous pouvez ne noter que ce qui change :

jsonc

```jsonc
"headings": {
  "1": { "size": 2.4, "color": { "light": "#0b3d91", "dark": "#7fb0ff" } },
  "2": { "size": 1.6, "indent": 0.5 }
},
"toc": {
  "3": { "indent": 2 }
}
```

> **Ne pas confondre** : `style.toc` = apparence du sommaire **latéral** (page HTML). Le bloc racine `toc` (§5.5) = table des matières **écrite dans le Markdown**.

Couleurs : `colors`. Par thème (`light` / `dark`), les clés suivantes (couleurs CSS : `"#1f2328"`, `"rgb(0 0 0)"`, `"steelblue"`) :

Clé	Désigne	Défaut clair / sombre
`bg`	fond de la page	`#ffffff` / `#111111`

Clé	Désigne	Défaut clair / sombre
`fg`	texte	`#1f2328` / `#f0f0f0`
`muted`	texte secondaire (en-têtes / pieds PDF)	`#59636e` / `#a0a0a0`
`border`	bordures, séparateurs	`#d1d9e0` / `#3d3d3d`
`sidebarBg`	fond du sommaire latéral	`#f6f8fa` / `#191919`
`accent`	liens, titre actif du sommaire, poignée de redimensionnement	`#0969da` / `#d0d0d0`
`codeBg`	fond du code, lignes paires des tableaux	`#f6f8fa` / `#191919`
`quote`	texte des citations	`#656d76` / `#a0a0a0`

jsonc

```jsonc
"colors": {
  "light": { "accent": "#c0392b", "sidebarBg": "#fff8f0" },
  "dark":  { "accent": "#ff8a65" }
}
```

`style.colors` est un réglage direct de compatibilité : ses valeurs remplacent les couleurs correspondantes de `pageTheme` pour le mode choisi. Pour un thème nommé réutilisable, préférez `pagePalettes` et `pageTheme`.

Aller plus loin (CSS). Pour tout ce que le `.jsonc` ne couvre pas :

Fichier	Portée
`template/style.css`	Apparence de la page HTML (**tous** les documents).
`template/print.css`	Mise en page du PDF (**tous** les documents).
`template/mermaid.css`	Style **des diagrammes Mermaid** (couleurs, bordures, polices des nœuds…), HTML et PDF.
`template/highlight.css`	Style **des balises de code** (couleurs des mots-clés, chaînes, commentaires, fonctions…), HTML et PDF.
`template/app.js`, `template/print.js`	Comportements de la page / du PDF (à éviter sauf besoin précis).

Fichier	Portée
`style.customCss`	Un CSS **propre à un document**, appliqué en dernier.

Exemple : `doc/exemple.jsonc` → `"customCss": "styles/document.css"` lit `doc/styles/document.css`. Après modification d'un fichier de `template/`, `npm run dev` recharge la page automatiquement.

5.8 Anciennes clés et erreurs fréquentes

- `source`, `mediaDirs`, `outputDir` et `pdf.output` **n'existent plus** : le document est déduit du nom (`exemple.md` ↔ `exemple.jsonc`), les médias sont lus dans `doc/`, les sorties vont dans `output_html/<nom>/` et `output_pdf/`. L'outil affiche un message explicite s'ils sont présents.
- `"… : clé inconnue"` : faute de frappe dans un nom de clé de `numbering`, `toc` ou `header` / `footer` (les clés valides sont listées).
- `JSON invalide` : virgule manquante ou en trop, guillemet oublié. Un `//` dans un texte entre guillemets est correctement ignoré (ce n'est pas un commentaire).
- Un niveau de titre doit être un entier de 1 à 6 ; `range` doit respecter `min ≤ max`.

---

6. Organisation des dossiers

```
.
├── doc/                  ← VOS documents (à modifier)
│   ├── exemple.md              le texte Markdown (exemple fourni : à
renommer)
│   ├── exemple.jsonc           sa configuration (même nom que le .md,
extension .jsonc)
│   └── media/                  vos images (partagées par tous les documents)
├── template/             ← le VISUEL des générations (CSS / JS), modifiable :
style.css, print.css, mermaid.css, highlight.css…
├── output_html/          ← généré : un sous-dossier par document, avec son
HTML + ses seuls médias utilisés
│   └── exemple/
│       ├── index.html
│       └── media/…
├── output_pdf/           ← généré : exemple.pdf
├── scripts/              ← l'outil lui-même (build, init, edit…), à ne pas
modifier
│   └── state/            ← mémoire de numérotation et manifestes HTML (JSON
internes, non versionnés)
├── package.json          ← liste des commandes npm
└── README.md
```

Règles importantes :

- Les fichiers générés sont mis à jour dans `output_html/` **et** `output_pdf/` : ne travaillez jamais dedans. Les anciens dossiers, s'ils existent, ne sont ni déplacés ni supprimés automatiquement.
- Un document n'est généré que si `doc/<nom>.md` **ET** `doc/<nom>.jsonc` existent (même `<nom>`). Un `.md` sans `.jsonc` est ignoré ; une config invalide est signalée et ignorée, les autres documents sont quand même traités.
- Plusieurs `.md` = des **variantes** d'un même sujet qui partagent le même dossier `doc/media/`.
- Les images se référencent **relativement au** `.md` : `![schéma](media/schema.png)`.
- Dans `output_html/<nom>/`, le livrable contient le HTML et un seul dossier `media/`. Les ressources référencées dans `doc/media/` gardent leur chemin ; les autres ressources locales (y compris les fichiers JSON liés) sont copiées dans `media/linked/`, et leurs URL sont réécrites dans le HTML. **Seules les ressources utilisées** sont copiées. Pour publier ou envoyer la page : donnez/ouvrez **tout le dossier** `output_html/<nom>/`.

- Les fichiers qui ne sont plus utilisés sont supprimés de `output_html/<nom>/` au build suivant (uniquement ceux que l'outil y avait écrits). Les manifestes JSON qui mémorisent ces fichiers sont conservés dans `scripts/state/`, pas dans les sorties HTML. Les ressources JSON explicitement référencées dans un Markdown restent des ressources de document et peuvent, elles, être copiées avec les autres médias.
- **Personnaliser l'apparence** : modifiez les fichiers de `template/` (voir §5) ; ils s'appliquent à tous les documents.

---

7. Dépannage

Problème	Solution
`node` / `npm` : « commande introuvable »	Node.js non installé, ou terminal ouvert avant l'installation : fermez et rouvrez-le.
`npm : Le terme « npm » n'est pas reconnu…` / `'npm' is not recognized`	Node.js n'est pas installé (ou terminal ouvert avant l'installation) : voir §1.1, puis rouvrez le terminal.
`Cannot find module …`	Lancez `npm install` dans ce dossier.
« aucun document à générer »	Vérifiez que `doc/<nom>.md` **et** `doc/<nom>.jsonc` existent avec le même nom.
« .jsonc : … document ignoré »	Erreur dans la config (virgule manquante, valeur invalide) : le message indique laquelle.
Images absentes dans le HTML	Ouvrez le HTML **depuis** `output_html/<nom>/` avec son dossier `media/` à côté, ou mettez `embedImages: true`.
« fichier situé hors de doc/ »	Une image est référencée avec `../` hors de `doc/` : déplacez-la dans `doc/`.
PDF : « navigateur introuvable »	Installez Chrome ou Edge, ou indiquez `pdf.browser` (chemin de l'exécutable).
`dev` : « port déjà utilisé »	Un autre `npm run dev` tourne encore (fermez-le) ou lancez avec un autre port : `$env:PORT=5000; npm run dev`.
Liens internes cassés signalés	Après renumérotation, relancez `npm run edit:numbering` (met à jour les liens) ou corrigez à la main.

8. Limites connues

- Les documents doivent être **directement dans** `doc/` (pas de sous-dossiers de documents) ; les médias, eux, peuvent être dans des sous-dossiers.
- Un titre numéroté dont le texte commence par un nombre (« 2024 bilan ») peut perdre ce nombre lors d'un changement de numérotation.
- Les diagrammes Mermaid en `elk` ajoutent environ 2,4 Mo au HTML.
- Les sauts de page automatiques ne peuvent pas être parfaits : une section plus longue qu'une page est forcément coupée.